

Module 3 Lab:

Code Coverage Testing Tool

JR Clark (002035589)

NEU CSYE6255 (Summer 2026)

Professor Omar Badreldin / TA Karthik Kashyap

PART 1: Tool Installation & Execution

At the start of this lab, I first used Terminal to confirm existing Java & Homebrew installations. Since I had previously installed these for a class in the previous term, I kept the existing versions though the Lab instructions specify JDK 17 or JDK 21. Next, I installed Maven and confirmed version installation from Terminal. Appendix 1 shows the commands executed and version outputs, which confirmed both Java and Maven were correctly configured on my system.

With the updated instructions from our TA in Canvas, I was able to successfully install IntelliJ IDEA without issues. My main challenges were environmental: a failing Dell Dock silently switched my internet connection from Ethernet to a slower 2.4Ghz wi-fi connection, and needed to resolve several configuration details in the IntelliJ IDEA app and pom.xml file to ensure that all my plug versions were aligned.

After completing the setup, I ran the mvn clean test from Terminal. Appendix 2 shows the Terminal output, including the Surefire execution report. JaCoco generated its HTML report successfully, and Appendix 3 shows the HTML report opened in Chrome. I also ran the built-in IntelliJ code coverage tool via the top menu using Run → Run with Coverage, and Appendix 4 shows the resulting HTML report. While I ran these tools separately, I also reviewed an article describing how JaCoCo can be integrated into IntelliJ IDEA and briefly explored the available configuration options.¹

¹ GeeksforGeeks. (2025, July 23). *JaCoCo with IntelliJ IDEA*. <https://www.geeksforgeeks.org/software-testing/jacoco-with-intellij-idea/>

PART 2: Coverage Metrics Analysis

JaCoCo provides more detailed coverage metrics than IntelliJ, including missed instructions, cyclomatic complexity, and highlighting where there are no branches vs calling it 100% coverage. This granularity allows a clearer view of how thoroughly the tests examine the code:

Table 1: Metrics Comparison: JaCoCo vs IntelliJ

	JaCoCo	IntelliJ
Class	100% (1/1)	100% (1/1)
Line Coverage	100% 5/5	100% (5/5)
Branch Coverage / Missed Branches	0 of 0	100% (0/0)
Method Coverage	100% 5/5	100% (5/5)
Missed Instructions	0 of 19 (100%)	N/A
Cyclomatic Complexity	5/5	N/A

Both tools confirm full coverage for this simple calculator code. IntelliJ analyzes source-level execution of the code written by the developer.² JaCoCo uses a more complex set of counters to calculate coverage metrics that includes Java byte code instructions.³ The identical results indicate that all test cases reached every part of the code. JaCoCo's additional metrics provide insight into internal code paths and potential branching logic. JaCoCo's metrics would become more informative as the codebase grows in complexity (ie, adding branches and involving more developers).

PART 3: Ease of Set Up and Use Comparison

IntelliJ IDEA provides the simplest coverage workflow, requiring only a single menu selection to run code coverage. The process is fully integrated into the IDE and does not require manual configuration. In contrast, JaCoCo required manual editing of pom.xml to include the

² JetBrains. (n.d.). Code coverage. JetBrains Help. <https://www.jetbrains.com/help/idea/code-coverage.html>

³ EclEmma. (n.d.). JaCoCo - Coverage counters. <https://www.eclEmma.org/jacoco/trunk/doc/counters.html>

Maven plugin, verifying version compatibility, and executing commands from Terminal. While IntelliJ's coverage tool is immediately available to use after installing IDEA, JaCoCo's setup demands a clearer understanding of Maven structure and plugin management, highlighting a more technical setup suited to automated build environments.

PART 4: Report Visualization Comparison

JaCoCo interface is more structured and provides clearer navigation that includes sortable columns, allows drilling down, and provides color-coded indicators. This makes it easy to understand exactly which parts of the code were tested. The IntelliJ report interface is minimal and intended for quick, in-IDE reference. It highlights lines directly in the editor and provides a simple summary window, but not a design made for sharing or detailed analysis outside of the IDE. This reflects the intended use of each tool: JaCoCo produces a full external report suitable for review and documentation, while IntelliJ focuses on immediate developer feedback during work in progress.

PART 5: Integration Assessment

IntelliJ's coverage is essentially single-developer, single-session tool meant to help developers check as they are working. JaCoCo's output files (HTML, CSV, XML) in `target/site/jacoco` can be picked up by CI/CD pipelines (Jenkins, SonarQube, etc), shared with team members or archived. Maven even outputs execution reports to the `surefire` folder, but IntelliJ coverage execution reports stay in the app unless you specifically take an extra step of manually generating an html report. This is done automatically by Maven in parallel with JaCoCo each time. Additionally, IntelliJ does not generate an external test execution report equivalent to Maven's Surefire report, which documents pass/fail results, test counts, and timing outside of the IDE.

PART 6: Detailed Findings

Both tools produced identical results: all five tests passed with zero failures, exceptions, or skipped tests. JaCoCo reported full line, method, and instruction coverage, with no missed instructions and a cyclomatic complexity value that matched the structure of the calculator code. IntelliJ also showed 100 percent line and method coverage. Because the code contains no branches, both tools reported complete branch coverage. The consistency across both tools confirms that the test suite exercised every part of the current implementation.

However, JaCoCo revealed that the 5 simple methods resulted in 19 byte code instructions, analysis that will increase in complexity as the codebase also increases in complexity. JaCoCo was also more precise – it reported “n/a” for branch coverage while IntelliJ reported “100% (0/0).”

PART 7: Actionable Insights

JaCoCo is better suited for team environments and automated build pipelines, while IntelliJ coverage is best suited for an individual developer working inside the IDE. The simplicity of the sample code and absence of branching logic result in very clean, uniform coverage across both tools. Adding conditional logic or additional methods to the Calculator would make branch coverage more meaningful and would highlight differences in how each tool reports uncovered paths. As the codebase grows, more complex tests, collaboration through Git, and CI integration would reveal gaps not visible in this small example and make JaCoCo’s detailed metrics more valuable.

APPENDIX 1: Terminal confirmation of Java & Maven

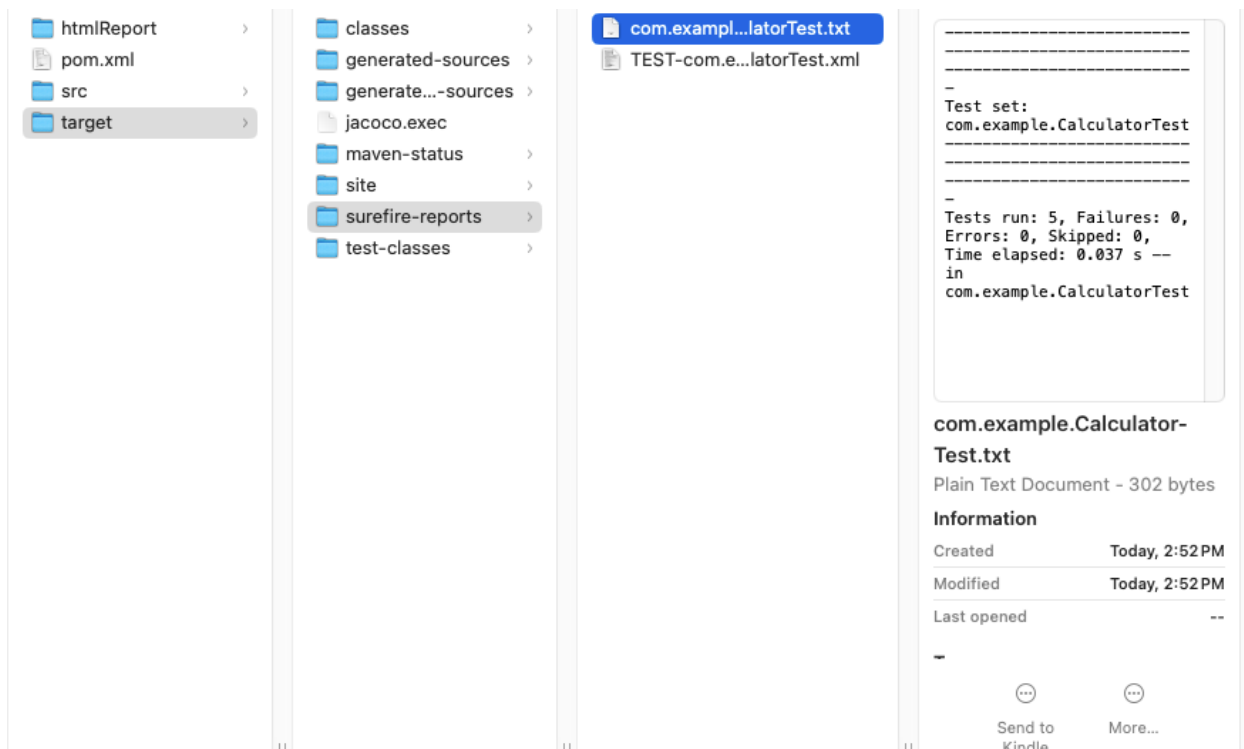
```
blackcatranch-m3@JRC-M3 ~ % mvn -v
Apache Maven 3.9.16 (2bdd9fddda4b155ebf8000e807eb73fd829a51d5)
Maven home: /opt/homebrew/Cellar/maven/3.9.16/libexec
Java version: 25.0.3, vendor: Eclipse Adoptium, runtime: /Library/Java/JavaVirtualMachines/temurin-25.jdk/Contents/Home
Default locale: en_US, platform encoding: UTF-8
OS name: "mac os x", version: "14.8.2", arch: "aarch64", family: "mac"
blackcatranch-m3@JRC-M3 ~ % java -version
openjdk version "25.0.3" 2026-04-21 LTS
OpenJDK Runtime Environment Temurin-25.0.3+9 (build 25.0.3+9-LTS)
OpenJDK 64-Bit Server VM Temurin-25.0.3+9 (build 25.0.3+9-LTS, mixed mode, sharing)
blackcatranch-m3@JRC-M3 ~ %
```

APPENDIX 2: Results of mvn clean output in terminal and Surefire execution report from Maven

Figure 1. mvn clean terminal output

```
[INFO]
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running com.example.CalculatorTest
Starting Calculator Tests
Divide by Zero Test -> Expect ArithmeticException
Add Test -> Expected: 5 | Actual: 5
Subtract Test -> Expected: 1 | Actual: 1
Divide Test -> Expected: 2 | Actual: 2
Multiply Test -> Expected: 6 | Actual: 6
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.037 s -- in com.example.CalculatorTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] --- jacoco:0.8.14:report (report) @ javacalculatorlab ---
[INFO] Loading execution data file /Users/blackcatranch-m3/Documents/javacalculatorlab/target/jacoco.exec
[INFO] Analyzed bundle 'javacalculatorlab' with 1 classes
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] -----
[INFO] Total time: 2.221 s
[INFO] Finished at: 2026-05-23T14:52:07-07:00
[INFO] -----
```

Figure 2. Directory structure showing execution reports folder and txt file in Mac OS finder:



Appendix 3: JaCoCo Test Results in HTML (3 levels)

javacalculatorlab > com.example

ClassesSessions

com.example

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
Calculator.java		100%		n/a	0	5	0	5	0	5	0	1
Total	0 of 19	100%	0 of 0	n/a	0	5	0	5	0	5	0	1

Created with JaCoCo 0.8.14.202510111229

File /Users/blackcatranch-m3/Documents/javacalculatorlab/target/site/jacoco/com.example/Calcul...

javacalculatorlab > com.example > Calculator

Calculator

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods
add(int, int)		100%		n/a	0	1	0	1	0	1
subtract(int, int)		100%		n/a	0	1	0	1	0	1
multiply(int, int)		100%		n/a	0	1	0	1	0	1
divide(int, int)		100%		n/a	0	1	0	1	0	1
Calculator()		100%		n/a	0	1	0	1	0	1
Total	0 of 19	100%	0 of 0	n/a	0	5	0	5	0	5

javacalculatorlab > com.example > Calculator.java

Calculator.java

```
1. package com.example;
2. public class Calculator {
3.
4.     public int add(int a, int b) {
5.         return a + b;
6.     }
7.
8.     public int subtract(int a, int b) {
9.         return a - b;
10.    }
11.
12.    public int multiply(int a, int b) {
13.        return a * b;
14.    }
15.
16.    public int divide(int a, int b) {
17.        return a / b;
18.    }
19. }
```


APPENDIX 4: IntelliJ IDEA test results in App

The screenshot displays the IntelliJ IDEA IDE interface for a project named 'javacalculatorlab'. The main window is divided into several panels:

- Project View:** Shows the project structure with folders like '.idea', '.vscode', 'htmlReport', 'src', 'main', and 'java'. The 'java' folder contains the package 'com.example' with 100% coverage.
- Code Editor:** Displays the source code for 'CalculatorTest.java'. The code defines a 'Calculator' class with methods: 'add', 'subtract', 'multiply', and 'divide'. Line numbers 1 through 20 are visible.
- Coverage Window:** Shows the coverage for 'CalculatorTest'. It includes a table with the following data:

Element	Class, %	Method, %	Line, %	Branch, %
com.example	100% (1/1)	100% (4/4)	100% (4/4)	100% (0/0)
Calculator	100% (1/1)	100% (4/4)	100% (4/4)	100% (0/0)

- Run Window:** Shows the execution of 'CalculatorTest'. It indicates that 5 tests passed out of 5 tests total, taking 10 ms. The test results are as follows:

- testDivideByZero() 8 ms
- testAdd() 2 ms
- testSubtract()
- testDivide()
- testMultiply()

The output of the tests is displayed in the Run window:

```
/opt/homebrew/opt/java/libexec/openjdk.jdk/Contents/Home/bin/java ...
Starting Calculator Tests
Divide by Zero Test -> Expect ArithmeticException
Add Test -> Expected: 5 | Actual: 5
Subtract Test -> Expected: 1 | Actual: 1
Divide Test -> Expected: 2 | Actual: 2
Multiply Test -> Expected: 6 | Actual: 6

Process finished with exit code 0
```

The bottom status bar shows the current file path: 'javacalculatorlab > src > test > java > com > example > CalculatorTest' and the encoding: '2:14 LF UTF-8 4 spaces'.

APPENDIX 5: IntelliJ Results in HTML

Current scope: all classes

Overall Coverage Summary

Package	Class, %	Method, %	Line, %
all classes	100% (1/1)	100% (5/5)	100% (5/5)

Coverage Breakdown

Package 	Class, %	Method, %	Line, %
com.example	100% (1/1)	100% (5/5)	100% (5/5)

generated on 2026-05-23 15:04

Current scope: [all classes](#) | [com.example](#)

Coverage Summary for Package: com.example

Package	Class, %	Method, %	Line, %
com.example	100% (1/1)	100% (5/5)	100% (5/5)

Class 	Class, %	Method, %	Line, %
Calculator	100% (1/1)	100% (5/5)	100% (5/5)

generated on 2026-05-23 15:04

Current scope: [all classes](#) | [com.example](#)

Coverage Summary for Class: Calculator (com.example)

Class	Class, %	Method, %	Line, %
Calculator	100% (1/1)	100% (5/5)	100% (5/5)

```
1 package com.example;
2 public class Calculator {
3
4     public int add(int a, int b) {
5         return a + b;
6     }
7
8     public int subtract(int a, int b) {
9         return a - b;
10    }
11
12    public int multiply(int a, int b) {
13        return a * b;
14    }
15
16    public int divide(int a, int b) {
17        return a / b;
18    }
19 }
```

generated on 2026-05-23 15:04

Methodology and Use of AI Tools

Google AI Mode, CoPilot and Claude were used to support the technical setup and configuration of the lab environment, including troubleshooting Maven and JaCoCo configuration. The analysis, comparisons, and written report reflect the author's own observations and conclusions.

Environment & Configuration:

- Confirming Java and Maven version compatibility with JaCoCo and IntelliJ
- Troubleshooting Maven compiler plugin version compatibility with JDK 25
- Resolving pom.xml structure for placing the JaCoCo plugin outside `<pluginManagement>`
- Verifying the latest stable version of maven-surefire-plugin

Running the Tools:

- How to run IntelliJ coverage tool via the top menu
- How to generate the JaCoCo HTML report via `mvn clean test`
- Understanding the difference between JaCoCo and Surefire outputs

Analysis & Comparison:

- Clarifying the difference between IntelliJ's Branch % and JaCoCo's Missed Branches
- Understanding what JaCoCo's cyclomatic complexity and instruction-level metrics mean
- Identifying which metrics are unique to JaCoCo vs IntelliJ
- Clarifying whether Surefire and JaCoCo are both generated by Maven

Report Writing:

- Reviewing draft sections against the rubric
- Generating an APA citations